

Assignment 5

Author: Lan Stare

Date: 2. 10. 2025

Question 1

(a):

The cost is **19**. The reason in this is that Kruskal's algorithm always finds the minimum spanning tree and with the algorithm described in (c) of this exercise, the weight of the spanning tree is 19. The explanation for this is also partially in (b).

(b):

The number of minimum spanning trees is **2**. This is quite hard to prove, but we can look at how many options each vertex has. Vertex A only has 1 option for the shortest path, so it chooses that one, vertices C, D, F and H also have a certain shortest edge. When we only add those to the spanning tree(s), we can see that there are no loops, so these edges are fixed. The vertex E has 2 possible options but both have already been added to the spanning tree(s) so we can move on. G already has 3 edges in the spanning tree so we can move on for now. The remaining vertex is B. It has 2 options to choose from. Both e(E, B) and e(F, B) are valid choices, since none of them form a cycle when chosen. In the last step of this explanation we have to choose the edge that connects the two subtrees and doesn't create a cycle. We have to choose the shortest edge, which is exactly e(F, G).

(c):

We start at vertex A meaning that we first choose the edge that connects A to another vertex and has minimum weight. In Kruskal's algorithm we first find the shortest edge (which in our case are both e(A, E) and e(E, F)) so we choose the edge e(A, E) and add A and E to the spanning tree. The shortest edge then is e(E, F) resulting in F being added to the spanning tree. From there I will choose the edge e(E, B) since it is among the shortest ones that are left, so I add B to the spanning tree. I will then choose e(G, H) and afterwards e(F, G), since none of these will result in a cycle. Our spanning tree now consists of vertices: A, B, E, F, G and H. On our next step we'll choose the edge e(G, C), adding C to the spanning tree, since it is our next shortest edge. The smallest remaining weights are 5, so we choose the one that connects the last remaining undiscovered vertex to the spanning tree: e(G, D). Our spanning tree has been formed and algorithm stopped, since we'd visited all vertices and not made any cycles.

Question 2

(a):

Vertex a is the first one added to the spanning tree, since it's our starting vertex.

- First iteration: vertex b is added to the spanning tree, since Prim's algorithm is greedy and chooses e(a, b) as the first edge because of the lowest weight (starting from a). The priority queue changes from [b, e, c] to [e, d, c] after b is added. Vertex a is the predecessor of b.
- Second iteration: vertex e is added because of priority, the new priority queue is: [c, d]. The predecessors: p(b) = a, p(e) = b
- Third iteration: vertex c is added because of priority, the new priority queue is: [d]. The predecessors: p(b) = a, p(e) = b, p(c) = e.
- Fourth iteration: vertex d is added because of being the only vertex left. The queue is empty. The predecessors: p(b) = a, p(e) = b, p(c) = e, p(d) = c.

(b)

We can imagine the n elements as n vertices and the weight of edges between them as the difference of their index in the sorted arrangement. To explain this, we can assign numbers 1 - n to vertices so that when the elements are sorted, their indexes are also sorted; 1, 2, 3, . . . n. We take the first element of the array and run Prim's algorithm on it. If the first element of the array is either 1 or n, the first and also each subsequent choice of path is obvious. In both cases you keep choosing the only edge (not yet in the spanning tree) as the next one. You either get a sorted array or its inverse. If the first element isn't 1 or n, let's say it's j, we have two options for the first path to take. Let's say we choose the path that takes us to the next biggest element and keep choosing the next biggest element until we reach n. At this point, we again want to choose the best option, which is to go back to element j and start choosing the next smallest element until we reach 1. At the end we still have to inverse the first j elements of the array, but we can do that in linear time. The same logic applies if we first choose the path that takes us to the next smallest element.

Question 3

For finding the Maximum spanning tree we can use the same idea as the one in Prim's or Kruskal's algorithm. Let's say we choose Prim's algorithm. The only difference is that on each step we select the edge of maximum weight instead of the one of minimum weight (like in Prim's). This way we form the spanning tree the same way as in the original Prim's algorithm, so the overall complexity is also the same. In the lectures we discussed that the complexity is $O((|V| + |E|) \log(|V|))$ (since it uses a priority queue like Dijkstra's algorithm).

Question 4

(a): FALSE

There exists a graph with more than |V| - 1 edges such that the heaviest edge has to be in the MST; counterexample: Graph G is undirected and has vertices A, B, C, D with edges and their weights: e(A, B) = 10, e(A, C) = 20, e(B, C) = 30, e(B, D) = 80. The graph has 4 edges and 4 nodes. The unique heaviest edge e(B, D) with weight 80. However this edge has to be included in the MST, since spanning trees have to visit every vertex and in our tree the vertex D is connected only to B with edge e(B, D) and weight 80. The MST has to therefore contain the heaviest edge.

(b): FALSE

A simple counterexample: We have an undirected graph with vertices A, B, C with edges and their weights: e(A, B) = 30, e(A, C) = 40, e(B, C) = 20. Dijkstra's algorithm starting at vertex A will take edges e(A, B) and e(A, C), making a spanning tree of weight 30 + 40 = 70, while Prim's algorithm will produce an MST with edges: e(A, B) and e(B, C) with the weight: 30 + 20 = 50. Therefore the Dijkstra's algorithm did not produce an MST. It produces an SPT (shortest-path tree).

(c): FALSE

If I understand the question correctly then this is a counter example: First we take an undirected graph G with vertices: A, B, C and weighted edges: e(A, B) = 10, e(A, C) = 10, e(B, C) = 50. A spanning tree containing the minimum edge e(A, B) can be the one that contains the edges e(A, B) and e(B, C) while it is obviously not an MST. An MST of this graph would have edges e(A, B) and e(A, C).

!! Please tell me if you would prefer me to upload a .jpeg picture of my handwritten counterexamples rather than me writing every edge and vertex down!!

Question 5

```
In [ ]: S_nad = ["D", "A", "B", "D", "A", "C", "D", "D", "A"]
S_pod = ["B", "C", "D", "A"]
S_nic = ["A", "B", "A", "D", "B"]

def is_sub_of_S(og, predp):
    """ We are using pointers for both lists. """
    i = j = 0
    while i < len(og) and j < len(predp):
        if og[i] == predp[j]:
            j += 1 #So for every j we scan through the first i elements of og (until we find value predp[j]) and then when we find it we can continue from from j + 1. So we only move on from i
            i += 1 #No matter if we find the right value or not we always add 1 to the index, so we never go through the same element in og.
        return j == len(predp) #

print(is_sub_of_S(S_nad, S_pod))
print(is_sub_of_S(S_nad, S_nic))
```

True

False

Correctness: We use two-pointer system to compare elements in og and predp. I have explained the way the function comparison works in the code comments, serving as my proof of correctness.

Complexity: og and predp are lists (they store their length), so the length is calculated in $O(1)$. In the next row, we are comparing elements, but because we only have to go through each element once (because we always start from the next element once we've found a matching element). So the complexity is $O(m + n)$, which is linear.